claude-windows / 96963dab-d38f-488c-af44-530ddbeefed1

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbeefed1\subagents\workflows\wf_00bebd3c-82f\agent-a766fb39ba708088e.jsonl`
- SHA-256: `8220962e6712ebe76a5ca67adae4508fd5ee8a6551f0e00f9e8ceb9ef74590c8`
- Source modified: `2026-07-08T05:50:34+00:00`
- Imported at: `2026-07-08T15:59:27+00:00`
- project: `wf_00bebd3c-82f`
- session_id: `96963dab-d38f-488c-af44-530ddbeefed1`

Transcript

1. user (2026-07-08T05:44:49.641Z)

You are mapping a subsystem of the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (Windows checkout; read-only ? do NOT edit anything).
Context: GitHub issue #524 wants to (a) move validation onto the L4 Cloud Run Job worker via a config-driven phase_placement knob, (b) drop a redundant re-hash in /api/process by trusting the upload's streamed video_id, and (c) design a topology where upload->stitch runs on the L4 with async copy-to-bucket. You are producing the precise code map a designer/implementer needs. Cite exact file paths and line numbers. Read the actual code, not just docs. Follow imports where the flow crosses files.
Subsystem: EVIDENCE + TESTS. (1) Read docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md section 9 (the 2026-07-08 live CUJ) and extract the full phase timeline with exact wall-clock numbers (upload duration/size, the ~30s pre-validation gap, validation 183s, dispatch/cold-start, detect 161.6s, ingest, compile ~12m55s, mirror) ? these are the design doc's evidence base. (2) Map the test layout for the areas #524 touches: which test files cover uploads.py, orchestration processing/dispatch, compute/cloud_run, validators, config models; how run_all_tests.py works, the coverage floor (--cov-fail-under value in CI config), and the mocking conventions (offline, fully mocked). Cite the CI workflow file for the exact gates (ruff/mypy/leak_scan/check_md_links).

2. user (2026-07-08T05:45:01.311Z)

DEPLOY_REPORT_cloud-serving_2026-07-04.md
DEPLOY_REPORT_cloud-serving_a1-cuj_2026-07-04.md
DEPLOY_REPORT_cloud-serving_a1-e2e_2026-07-04.md
DEPLOY_REPORT_cloud-serving_alpha-all1v1_2026-07-06.md
DEPLOY_REPORT_cloud-serving_cp-rebuild_2026-07-05.md
DEPLOY_REPORT_cloud-serving_gpu-resident-nvdec_2026-07-07.md
DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md
README.md
RUNLOG_cloud-serving_exec-status-fastfail_2026-07-07.md
RUNLOG_truly-local_TEMPLATE.md

3. user (2026-07-08T05:45:06.457Z)

run_all_tests.py:10:   python run_all_tests.py --cov=backend --cov-report=term-missing   # coverage
.github\workflows\nightly.yml:141:   run: python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70
.github\workflows\ci.yml:146:   run: python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70

4. user (2026-07-08T05:45:14.452Z)

139 C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-

indexer/docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md

5. user (2026-07-08T05:45:15.135Z)

```
1:# DEPLOY REPORT ? cloud-serving, validation-latency closure (#512 shared-decode + #513 worker skip-revalidation), 2026-07-07
5:## 1. TL;DR
18:## 2. Resources
24:## 3. Exact commands
26:# --- Provenance: prove the live image carries both fixes ---
32:# --- #513 proof: worker A/B on the LIVE image (Video 1 already staged in the serving bucket) ---
34:# Run A ? WITH --skip-validation (exactly what orchestration._maybe_dispatch_remote emits on dispatch):
37:# Run B ? WITHOUT --skip-validation (worker re-validates with the NEW #512 validators ? still blur-rejects):
41:# --- #512 timing: bracket "pulled ? -> in.mp4"  ?  "validation FAILED/SKIPPED"  ?  "Phase 2 (WASB) completed" ---
45:# --- #512 off-box A/B (isolated clones; run_all decodes the SAME sample indices in both trees) ---
51:## 4. Run results
87:## 5. Rollout state
92:## 6. Honest limits / not-verified
99:## 7. Cost
102:## 8. Closure verdict
105:## 9. Update (2026-07-08) ? live SPA CUJ closes the two owner-gated cells (on rev `00019`)
```

6. user (2026-07-08T05:45:23.196Z)

```
105     ## 9. Update (2026-07-08) ? live SPA CUJ closes the two owner-gated cells (on rev `00019`)
106
107     The two ? cells in §4a (the **live control-plane validation wall** and the **end-to-end reel**) were owner-gated on the Cloudflare-Access `POST /api/process`. On **2026-07-08** the owner drove one real SPA upload (compute=**Cloud**) end-to-end, so both are now measured on live serving infra. Mirrors the gpu-resident report's "§5 Update" append ? the body above is unchanged; this section records what the live run added.
108
109     **Two honest deltas from the recipe** (state moved after this report's 2026-07-07 image cutover):
110     1. **Live rev rolled `00018` ? `00019`.** Traffic is 100% on **`rally-control-plane-00019-96f`**, image `rally-worker@sha256:c6393d27b4ae?` (tag `:latest`), deployed 2026-07-07T17:06Z ? ~51 min after the `00018`/`8045f54e` cutover §2 documents; the worker Job is on the same `:latest`=`c6393d27` digest (both surfaces match). It's an undocumented forward-bump, but #512 (`20862d7`) and #513 (`23c1868`) are ancestors of `origin/master`, `f5c9dd9` (the `00018` src) is an ancestor of master, and the run **empirically** shows both fixes live (one-pass validation wall + worker `validation SKIPPED`). Measured on what is actually serving.
111     2. **A fresh clip, not Video 1.** The upload was **"First CUJ.MP4"** ? **1.83 GiB** (1,960,043,219 B), source duration **174.17 s** (?84 Mbps, 4K-class ? *heavier* than Video 1's 1.41 GiB), video_id `2f31b45e?`. So the live wall **corroborates** the Video-1 projection on a comparably-heavy clip; it is **not** a re-measurement of Video 1's 412 s?~170 s A/B (a different clip has a different absolute wall).
112
113     **Full phase breakdown (UTC 2026-07-08, rev `00019`):**
114
115     | # | Phase | Window | Wall | Machine | Notes |
116     |---|---|---|---|---|---|
117     | 1 | Upload (client?server, 80-part multipart) | 04:25:56 ? 04:28:51 | **~2m55s** | client uplink | 1.83 GiB @ ~84 Mbps ? owner bandwidth, not infra |
118     | 2 | Accept ? validation start (stage) | 04:28:52 ? 04:29:22 | ~30 s | control-plane | stage upload to serving bucket |
119     | 3 | **Validation (#512 shared-decode)** ? | 04:29:22 ? 04:32:25 | **183 s (3m03s)** | control-plane 2 vCPU | one-pass scene_cut/blur/relevance; PASSED |
120     | 4 | Dispatch ? worker ready (cold start) | 04:32:25 ? 04:33:20 | ~55 s | Cloud Run Job
```

```
| image pull + GCSFuse mount + torch/cuda init |
121     | 5 | **Worker skip-validation (#513)** ? | 04:33:20 ? 04:33:24 | ~4 s | L4 |
`validation SKIPPED`, 0 re-validation |
122     | 6 | Detection / segment (WASB) | 04:33:24 ? 04:36:20 | **161.6 s** | L4 (cuda) | 15
candidate windows, 2561 visible pts |
123     | 7 | Ingest + finalize | 04:36:20 ? 04:36:24 | ~4 s | control-plane | intervals?DB,
`status=success` |
124     | ? | (user reviews rallies, clicks Compile) | 04:36:24 ? 04:37:26 | ~62 s idle | ? |
human-in-the-loop |
125     | 8 | **Compile / stitch** ? slow | 04:37:26 ? 04:50:21 | **~12m55s** | control-plane 2
vCPU | 11 clips, ~67 s/clip 4K re-encode+watermark; concat ~2 s; mirror ~11 s |
126
127     Server pipeline (process?success, excl. upload+idle): **7m32s**. Full CUJ (upload?reel):
**~24m25s** wall.
128
129     **Cell 1 ? control-plane validation AFTER (live):** **183 s (3m03s)** on rev `00019`,
PASSED ? the #512 shared-decode validators on the real 2-vCPU serving path, in the projected
**2?3 min** band on a clip *heavier* than Video 1 (pre-#512 per-sample seeks were the
~6m52s-class baseline). Evidence: `Running validations 04:29:22.789 ? Video passed checks
04:32:25.780`.
130
131     **Cell 2 ? end-to-end reel (live):** `/api/compile` stitched **11 of the 15** detected
rallies (0.5 s pad, per-clip branding watermark) from the 174.17 s source ? **`gs://khelsutra-
rally-serving/highlights/2f31b45e?/first-cuj-ks-highlight-reel.mp4`** (**581.84 MiB**, saved
04:50:10 *"Dynamic highlight reel saved (581.8 MB)"*, mirrored 04:50:21, served via
`/api/highlights` 307). **Reel ? produced end-to-end on the live control-plane** ? source fetch
? per-clip trim+watermark ? concat ? mirror-upload, all exercised.
132
133     **Bonus ? #513 proof upgraded** from the §6 dispatch-faithful *direct exec* to a genuine
**control-plane-initiated dispatch** (`rally-worker-64bhj`): `_maybe_dispatch_remote` emitted
`--skip-validation` on a live SPA run and the worker honored it (0 re-validation).
134
135     **? Finding ? stitch is the scaling bottleneck (follow-ups filed).** The compile ran
**~13 min on the CPU-only 2-vCPU control-plane** (~45?90 s per ~7 s clip: the watermark forces a
full 4K re-encode, no NVENC). With validation (183 s) *also* on the control-plane and serialized
by `_LOCAL_COMPUTE_LANE`, one run pins the control-plane for ~16 min of heavy CPU ? it won't
scale past a few concurrent users. Filed: **#521** (move compile/stitch to the L4 worker + make
phase?machine placement config-driven) and **#520** (step-level debug tracing, server+UI, to a
low-latency sink ? this breakdown had to be hand-correlated across 3 log streams).
136
137     **Cost (heuristic):** one L4 detect exec (`rally-worker-64bhj`, ~222 s) ? **$0.15?0.55**
(rate-dependent, dominant) + control-plane CPU for validation+stitch (~960 s @ 2 vCPU) ?
**~$0.05** + reel egress (581 MB, if downloaded to internet) ? **~$0.07** + storage/ops **<
$0.02** + Gemini **$0** (skip_ai_handoff). **? $0.3?0.7 total**, ~one L4 exec as projected ? but
the stitch's real cost is **concurrency** (~16 min of control-plane heavy CPU), not dollars.
138
139     **Net:** both owner-gated cells are now live-measured on rev `00019` ? validation wall
**183 s** (corroborating the 2?3 min projection) and the reel compiles end-to-end. #512 one-pass
validation and #513 skip-validation-on-real-dispatch are now both directly observed in prod. The
Video-1 412 s?~170 s A/B remains a projection by design (different, heavier clip).
140
```

7. assistant (2026-07-08T05:45:30.875Z)

Section 9 fully captured. Now mapping the source areas and their tests.

8. user (2026-07-08T05:45:34.004Z)

```
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/api:
__init__.py
__pycache__
auth.py
job_worker.py
jobs.py
static
```

```
uploads.py

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/compute:
__init__.py
__pycache__
base.py
cloud_run.py

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/config:
__init__.py
__pycache__
loader.py
models.py
presets.py
startup.py
```

9. user (2026-07-08T05:45:34.627Z)

```
__init__.py
__pycache__
test_ablation.py
test_annotation_providers.py
test_annotation_proxy.py
test_api_annotations.py
test_api_assets.py
test_api_cloud_dispatch.py
test_api_endpoints.py
test_app_home.py
test_app_launcher.py
test_async_jobs.py
test_audio_features.py
test_automated_provider.py
test_base.py
test_billing.py
test_bind_and_host.py
test_calibrate_local.py
test_calibrate_wasb.py
test_calibration.py
test_check_md_links.py
test_cleanup_caches.py
test_compiler.py
test_compute_backend.py
test_config.py
test_config_deployment.py
test_config_wasb.py
test_court_gate.py
test_database.py
test_db_upgrade_safety.py
test_deploy_manifests.py
test_detector_keying.py
test_device_context.py
test_distill_local.py
test_doctor.py
test_drain_concurrency.py
test_edge_auth.py
test_eval_annotations.py
test_eval_batch.py
test_eval_classifier.py
test_eval_harness.py
test_eval_metrics.py
test_eval_partial_labels.py
test_eval_regression.py
test_eval_stats.py
test_eval_training.py
test_execution_policy.py
```

```
test_experiment.py
test_experiment_honest.py
test_export_wasb_segments.py
test_ffmpeg_resolver.py
test_ffmpeg_timeouts.py
test_flywheel.py
test_frame_sampler.py
test_freespace.py
test_fusion_compare.py
test_fusion_features.py
test_fusion_golden_telemetry.py
test_fusion_hybrid.py
test_gemini.py
test_gemini_refine.py
test_generate_highlights.py
test_geometry.py
test_golden_fixtures.py
test_golden_fixtures_split.py
test_golden_manifest.py
test_golden_real_fixtures.py
test_gt_loader.py
test_hashing.py
test_hit_detector.py
test_import_smoke.py
test_input_source.py
test_job_self_scheduling.py
test_job_worker_loop.py
test_job_worker_module_seam.py
test_leak_scan.py
test_light_tier_no_torch.py
test_locale_analytics.py
test_logging_setup.py
test_main_module_budget.py
test_motion_segmenter.py
test_native_wasb_runner.py
test_nightly_reelcount.py
test_nightly_regression.py
test_output_base.py
test_p2b_install.py
test_path_safety.py
test_per_user_eval.py
test_per_user_gate.py
test_per_user_regression.py
test_person_detector.py
test_personalization_serving.py
test_profile_parity.py
test_promotion.py
test_provider_gemini.py
test_providers.py
test_rally_gate.py
test_rally_seg_eval.py
test_rally_seg_eval_report.py
test_rally_seq_proto.py
```

10. user (2026-07-08T05:45:41.448Z)

```
test_per_user_regression.py
test_person_detector.py
test_personalization_serving.py
test_profile_parity.py
test_promotion.py
test_provider_gemini.py
test_providers.py
test_rally_gate.py
test_rally_seg_eval.py
```

```
test_rally_seg_eval_report.py
test_rally_seq_proto.py
test_rally_slicer.py
test_rally_summary.py
test_redact.py
test_reingest_safety.py
test_reliability_hardening.py
test_reporting.py
test_reporting_harvest.py
test_reporting_spec_faq.py
test_restart_recovery.py
test_results.py
test_run_signals.py
test_run_telemetry.py
test_score_calibration.py
test_segmentation_metrics.py
test_serve_contrast.py
test_served_gate_a_regression.py
test_server_entrypoint.py
test_single_instance.py
test_splits.py
test_sport_profiles.py
test_startup_checks.py
test_storage.py
test_storage_capabilities.py
test_storage_gdrive_api.py
test_stub_runner.py
test_submit_md5_cache.py
test_tolerance_metrics.py
test_tracknet_hybrid.py
test_tracknet_runner.py
test_training_gen0.py
test_trajectory_hybrid_equivalence.py
test_ui_bundle.py
test_upload_download.py
test_upload_gcs_streaming.py
test_upload_retention.py
test_validation_cache.py
test_validators_base.py
test_variant_tuning.py
test_video.py
test_wasb_cache.py
test_wasb_nvdec_resident.py
test_wasb_runner.py
test_watermark.py
test_watermark_i18n.py
test_windowing.py
test_worker.py
test_workspace.py
test_wsl_tilde_quoting.py
test_yolo_hybrid.py
```

11. user (2026-07-08T05:45:42.187Z)

```
Found 3 files
tests\test_upload_gcs_streaming.py
tests\test_main_module_budget.py
tests\test_upload_download.py
```

12. user (2026-07-08T05:45:50.448Z)

```
backend\api\uploads.py:97:    Remote staging is selected by the same input config /api/process
resolves against
backend\api\uploads.py:288:    """Finalize: verify part count, move into place, return {path,
video_id} for /api/process."""
```

```
backend\api\uploads.py:301:        # part IS compute_video_id's digest (same bytes, same
algorithm), so /api/process
backend\api\uploads.py:318:            "next": "POST /api/process with this path",
backend\api\uploads.py:334:        "next": "POST /api/process with this path",
backend\config\startup.py:106:    # (1) The path-jail is load-bearing once authenticated tenants
can hit /api/process: WITHOUT it,
backend\config\startup.py:116:                "unset ? an authenticated tenant could make
/api/process read an arbitrary local file. "
backend\api\jobs.py:83:    """The whole /api/process submit decision (#448 dedup + #484 submit-
time cache) in one hook.
backend\api\job_worker.py:137:        # v8 compute-picker: the worker runs _execute_processing ?
_maybe_dispatch_remote, which
backend\api\job_worker.py:158:    # original /api/process pipeline). Both record their terminal
state the same way; only process
backend\api\job_worker.py:364:        # Same normalization the execution path applies
(_maybe_dispatch_remote lowercases
backend\config\models.py:381:    # When set, /api/process video_path must resolve under this
directory (hosted/tenant
backend\config\models.py:388:    # contain upload_dir or /api/process will reject the uploaded
paths.
backend\config\models.py:398:    # path is unchanged). When True, /api/process requires +
verifies the `Cf-Access-Jwt-Assertion`
backend\api\static\app.js:88:            const response = await fetch("/api/process", {
backend\infrastructure\database.py:266:        # /api/process request. `status` is the EXECUTION
state of the job
backend\infrastructure\database.py:464:        # is now 202+poll like /api/process so a slow
ffmpeg stitch can't trip the edge 524). `params`
backend\infrastructure\database_jobs.py:204:        """Dedup key for /api/process idempotency:
the {id, status} of an already-ACTIVE
backend\orchestration.py:4:Cloud Run dispatch + ingest path (``_maybe_dispatch_remote`` /
``_ingest_remote_segments``),
backend\orchestration.py:161:# guardrail in _maybe_dispatch_remote fails such a request FAST
instead of the cryptic deep
backend\orchestration.py:245:    """The /api/process result for a video whose detection already
exists ? ONE shape for
backend\orchestration.py:323:    """A ready-to-serve /api/process result when this video's
detection ALREADY exists ?
backend\orchestration.py:423:def _maybe_dispatch_remote(
backend\orchestration.py:777:    # _maybe_dispatch_remote when it dispatches. None ? nothing ran
(validation/config fail).
backend\orchestration.py:906:        remote = _maybe_dispatch_remote(
backend\main.py:241:    _maybe_dispatch_remote,
backend\main.py:268:    # default (deployment.compute_target). Only honoured for these two
values; see _maybe_dispatch_remote.
backend\main.py:681:    Edge-auth mirrors /api/process (default-OFF ? no-op locally).
``video_id`` is validated as a bare
backend\main.py:748:    Edge-auth mirrors ``/api/process``. Exactly one source: an
``uploaded_path`` (a local file from
backend\main.py:904:@app.post("/api/process", status_code=202)
backend\main.py:905:def process_video(req: ProcessRequest, request: Request):
backend\main.py:1102:    Async, mirroring /api/process: the ffmpeg stitch (trim + re-encode each
clip + concat + watermark)
backend\worker\__main__.py:280:    # --skip-validation (set by the dispatcher in
orchestration._maybe_dispatch_remote) skips it; the
backend\pipeline\segmenters\motion.py:23:    def process_video(
backend\pipeline\segmenters\base.py:44:    def process_video(
backend\pipeline\segmenters\trajectory_hybrid.py:174:    def process_video(  # type:
ignore[override]
backend\pipeline\segmenters\gemini.py:622:    def process_video(
backend\pipeline\segmenters\yolo_hybrid.py:486:    def process_video(  # type: ignore[override]
```

13. user (2026-07-08T05:45:51.302Z)

```
Found 15 files
tests\test_validation_cache.py
```

```
tests\test_worker.py
tests\test_api_cloud_dispatch.py
tests\test_compute_backend.py
tests\test_restart_recovery.py
tests\test_submit_md5_cache.py
tests\test_api_endpoints.py
tests\test_config_deployment.py
tests\test_deploy_manifests.py
tests\test_main_module_budget.py
tests\test_upload_download.py
tests\test_drain_concurrency.py
tests\test_startup_checks.py
tests\test_profile_parity.py
tests\test_billing.py
```

14. user (2026-07-08T05:45:58.663Z)

```
#!/usr/bin/env python
"""Run the FULL test suite for the badminton-highlight-indexer.

A single, discoverable entry point so you (or a reviewer / CI) can verify the
whole suite before opening a PR:

    python run_all_tests.py             # full suite, quiet
    python run_all_tests.py -v          # verbose (passthrough to pytest)
    python run_all_tests.py -k config   # filter by keyword (passthrough)
    python run_all_tests.py --cov=backend --cov-report=term-missing   # coverage

Anything after the script name is passed straight through to pytest.

Notes
-----
- The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
- The observability **reporting** tests skip cleanly if the optional ``obsreport``
  package isn't installed (they use ``pytest.importorskip``). To exercise them too:
      pip install -e ../sports-obsreport --no-build-isolation
- Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
"""

from __future__ import annotations

import importlib.util
import subprocess
import sys


def _installed(module: str) -> bool:
    try:
        return importlib.util.find_spec(module) is not None
    except (ImportError, ValueError):
        return False


def main() -> int:
    args = list(sys.argv[1:])
    # Default to quiet unless the caller chose a verbosity.
    if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
        args = ["-q", *args]
    cmd = [sys.executable, "-m", "pytest", *args]

    obs = (
        "available"
        if _installed("obsreport")
        else "NOT installed ? reporting tests will SKIP"
    )
```

```
49          bar = "=" * 72
50          print(bar)
51          print(" Full test suite:  " + " ".join(cmd))
52          print(f" obsreport (optional reporting dep): {obs}")
53          print(bar, flush=True)
54
55          rc = subprocess.run(cmd).returncode
56
57          print(bar)
58          print(
59              " RESULT: "
60              + ("PASS - all tests green" if rc == 0 else f"FAIL - pytest exit {rc}")
61          )
62          print(bar)
63          return rc
64
65
66      if __name__ == "__main__":
67          raise SystemExit(main())
68
```

15. user (2026-07-08T05:46:01.240Z)

```
Exit code 2
ci.yml
nightly.yml
```

16. user (2026-07-08T05:46:10.161Z)

```
No files found
```

17. user (2026-07-08T05:46:11.570Z)

```
mypy.ini
pyproject.toml
pytest.ini
ruff.toml
```

18. user (2026-07-08T05:46:17.014Z)

```
1       name: CI
2
3       on:
4         push:
5           branches: [master]
6         pull_request:
7
8       permissions:
9         contents: read
10
11      concurrency:
12        group: ci-${{ github.ref }}
13        cancel-in-progress: true
14
15      jobs:
16        # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
17        # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
18        # tracked .py, including scripts/ and training/, which the backend-only
19        # py_compile smoke sweep does not reach.
20        lint:
21          name: Ruff lint
22          runs-on: ubuntu-latest
23          timeout-minutes: 10
24          steps:
```

```
25            - uses: actions/checkout@v6
26            - uses: actions/setup-python@v6
27              with:
28                python-version: "3.13"
29            - name: Install ruff
30              # Pinned so an upstream ruff release can't redden every PR overnight; bump
deliberately.
31              run: python -m pip install ruff==0.15.16
32            - name: Ruff check
33              run: ruff check . --output-format github
34
35        # PII / attribution leak guard (tools/leak_scan.py): fails on a committed 32-hex MD5
36        # (the video_id shape) or a personal absolute user path in the shipped-code surface
37        # (backend/training/scripts/tools/eval_baselines; tests/ excluded ? legit fixtures).
Real
38        # names are scanned only via a gitignored private list locally; CI runs the structural
guard.
39        leak-scan:
40          name: Leak scan (PII / attribution guard)
41          runs-on: ubuntu-latest
42          timeout-minutes: 10
43          steps:
44            - uses: actions/checkout@v6
45            - uses: actions/setup-python@v6
46              with:
47                python-version: "3.13"
48            - name: Leak scan
49              run: python -m tools.leak_scan
50
51        # Internal-link guard (tools/check_md_links.py): fails on a broken internal markdown
link
52        # (missing file or dead #anchor) in a LIVE doc. docs/archives/** is excluded ? frozen
53        # snapshots whose relative links are historical-by-design (see docs/DOC_STATUS.md §2).
54        link-check:
55          name: Markdown link check (live docs)
56          runs-on: ubuntu-latest
57          timeout-minutes: 10
58          steps:
59            - uses: actions/checkout@v6
60            - uses: actions/setup-python@v6
61              with:
62                python-version: "3.13"
63            - name: Check internal markdown links
64              run: python -m tools.check_md_links
65
66        # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
67        # per-module quarantine list ? ratchet by removing entries as modules clean
68        # up. Typed core deps are installed so mypy sees their real signatures
69        # (without them ignore_missing_imports would Any-stub half the checks).
70        typecheck:
71          name: Mypy (lenient baseline)
72          runs-on: ubuntu-latest
73          timeout-minutes: 10
74          steps:
75            - uses: actions/checkout@v6
76            - uses: actions/setup-python@v6
77              with:
78                python-version: "3.13"
79            - name: Install mypy + typed core deps
80              # mypy pinned alongside ruff so a type-checker release can't redden CI
overnight.
81              run: python -m pip install mypy==2.1.0 pydantic fastapi uvicorn python-dotenv
google-genai numpy
82            - name: Mypy
83              run: mypy backend training
```

```
84
85        # Fast guard against the 7fbbeb0 class of breakage: a SyntaxError or
86        # module-level import error shipping while the suite stays green.
87        # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
88        # on machines without the GPU stack (test_import_smoke stubs what's absent).
89        import-smoke:
90          name: Syntax sweep + import smoke (no GPU stack)
91          runs-on: ubuntu-latest
92          timeout-minutes: 10
93          steps:
94            - uses: actions/checkout@v6
95            - uses: actions/setup-python@v6
96              with:
97                python-version: "3.13"
98            - name: Install minimal deps
99              run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai
100           - name: Run import smoke tests
101             run: python -m pytest tests/test_import_smoke.py -v
102
103       full-suite:
104         name: Full test suite (offline, mocked)
105         runs-on: ubuntu-latest
106         timeout-minutes: 30
107         steps:
108           - uses: actions/checkout@v6
109           - uses: actions/setup-python@v6
110             with:
111               python-version: "3.13"
112               cache: pip
113           - name: System libs for opencv-python
114             run: |
115               # Drop the runner's pre-installed Microsoft apt repo (azure-cli): it
periodically
116               # 403s on InRelease and aborts `apt-get update` under the `-e` shell, failing
the
117               # build on an unrelated repo we don't use. libgl1/libglib2.0 are in Ubuntu's
archive.
118               sudo rm -f /etc/apt/sources.list.d/*microsoft* /etc/apt/sources.list.d/*azure*
119               sudo apt-get update
120               sudo apt-get install -y libgl1 libglib2.0-0t64 || sudo apt-get install -y
libgl1 libglib2.0-0
121           - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
122             # torch/torchvision are NOT in pyproject deps ? their CPU/CUDA wheels
123             # need an out-of-band index that PEP 621 can't express. Keep installing
124             # them separately here, before the editable install pulls the rest.
125             run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
126           - name: Install trackers (git-only dep ? PyPI wheel is broken)
127             # D13/P12: ByteTrack moved out of `supervision` (removed in 0.30.0) to the
128             # standalone `trackers` package. Its PyPI wheel ships metadata only (no
129             # source ? roboflow/trackers#474), so it MUST come from git. PEP 621 can't
130             # express a git URL, so it's not in pyproject.toml (same pattern as torch).
131             # Installed here so the two D13/P12 tests in tests/test_person_detector.py
132             # actually run in CI instead of skipping via pytest.importorskip.
133             run: python -m pip install -r requirements-trackers.txt
134           - name: Install package (editable) + dev tooling
135             # Editable PEP 621 install: runtime deps + the `dev` group
136             # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
137             run: python -m pip install -e .[dev]
138           # obsreport (private, soft dep) is absent here: reporting tests skip via
139           # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
140           # is therefore calibrated against THIS lane's number (local runs with
141           # obsreport installed measure a little higher).
142           # Floor calibrated 2026-06-11 against this lane's measured 72% (local
```

```
143           # with obsreport: 73%). Ratchet it UP as coverage grows; never down
144           # without an owner decision.
145           - name: Run full suite (with coverage floor)
146             run: python run_all_tests.py --cov=backend --cov=training --cov-report=term
--cov-fail-under=70
147
148       # Cross-OS determinism guard for the committed golden regression fixtures
149       # (docs/GOLDEN_REGRESSION_FIXTURES.md). The CONTROL replay path is pure-stdlib
150       # (only numpy, no cv2/torch), so this is a FAST, focused job ? it does NOT run
151       # the full suite x3 OSes. It replays the Windows/Linux/macOS runner against the
152       # committed (parse-compared) snapshots, catching any cross-OS float drift beyond
153       # the 1e-6 tolerance before it can flake a contributor's PR on another machine.
154       golden-crossos:
155         name: Golden fixtures cross-OS (${{ matrix.os }})
156         strategy:
157           fail-fast: false
158           matrix:
159             os: [ubuntu-latest, windows-latest, macos-latest]
160         runs-on: ${{ matrix.os }}
161         timeout-minutes: 15
162         steps:
163           - uses: actions/checkout@v6
164           - uses: actions/setup-python@v6
165             with:
166               python-version: "3.13"
167           - name: Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
168             run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai numpy
169           - name: Golden-fixtures characterization (cross-OS determinism guard)
170             # Also runs the freeze-real / time-origin-reset / M2 refusal-gate tests
(synthetic /
171             # tmp_path only ? no GPU/network) so the reset metric-invariance (fix #2) and
the
172             # de-attribution paths get cross-OS coverage and surface any win/mac float
drift.
173             run: |
174               python -m backend.eval.golden_fixtures --check
175               python -m pytest tests/test_golden_fixtures.py
tests/test_golden_real_fixtures.py -q
176
177       # Commercial-clean licensing guardrail (MIT/Apache/BSD only ? PACKAGING_PLAN.md P0d).
Scans the
178       # RUNTIME dependency closure (`pip install .` ? no torch/dev) and fails on any
copyleft
179       # (GPL/AGPL/LGPL/SSPL). torch (BSD-3) is installed out-of-band and is not in this
closure. This
180       # automates a guardrail that was human-only; the report is printed for review either
way.
181       # D13/P12 carve-out: `trackers` (Apache-2.0; ByteTrack's new home) is also git-only
and thus
182       # absent from this pyproject closure ? UNguarded here, same shape as torch. It's
Apache-2.0 and
183       # depends only on the still-declared `supervision` (MIT), so the closure is clean
today; if
184       # `trackers` ever pulls a new transitive dep, this gate won't catch it ? audit on
bump.
185       license-scan:
186         name: Dependency license scan (no copyleft in runtime deps)
187         runs-on: ubuntu-latest
188         timeout-minutes: 15
189         steps:
190           - uses: actions/checkout@v6
191           - uses: actions/setup-python@v6
192             with:
193               python-version: "3.13"
```

```
194           - name: Install runtime deps + pip-licenses
195             run: |
196               python -m pip install .
197               python -m pip install pip-licenses
198           - name: Report dependency licenses
199             run: python -m piplicenses --format=markdown --with-urls --order=license
200           - name: Fail on any copyleft (GPL/AGPL/LGPL/SSPL) in the runtime closure
201             run: |
202               if python -m piplicenses --format=plain | grep -iE "GPL|SSPL"; then
203                 echo "::error::A runtime dependency carries a copyleft (GPL/AGPL/LGPL/SSPL)
license ? the guardrail is MIT/Apache/BSD only. Review the report above."
204                 exit 1
205               fi
206               echo "OK: no GPL/AGPL/LGPL/SSPL in the runtime dependency closure."
207
208       # Built-wheel smoke (PACKAGING_PLAN.md P0d): build the wheel, install it NON-editable
from a
209       # non-repo cwd, and launch the console script. Catches packaging regressions the
source suite
210       # can't ? namespace-package resolution (backend has no __init__.py), missing data
files, hidden
211       # imports. (Validated locally: a clean-venv wheel install + CPU torch runs `indexer
--help`.)
212       build-artifact:
213         name: Built wheel smoke (install + launch)
214         runs-on: ubuntu-latest
215         timeout-minutes: 30
216         steps:
217           - uses: actions/checkout@v6
218           - uses: actions/setup-python@v6
219             with:
220               python-version: "3.13"
221           - name: System libs for opencv-python
222             run: |
223               # Drop the runner's pre-installed Microsoft apt repo (azure-cli): it
periodically
224               # 403s on InRelease and aborts `apt-get update` under the `-e` shell, failing
the
225               # build on an unrelated repo we don't use. libgl1/libglib2.0 are in Ubuntu's
archive.
226               sudo rm -f /etc/apt/sources.list.d/*microsoft* /etc/apt/sources.list.d/*azure*
227               sudo apt-get update
228               sudo apt-get install -y libgl1 libglib2.0-0t64 || sudo apt-get install -y
libgl1 libglib2.0-0
229           - name: Build the wheel
230             run: |
231               python -m pip install build
232               python -m build --wheel
233           - name: Install the built wheel + CPU torch (out-of-band index)
234             run: |
235               python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
236               python -m pip install dist/*.whl
237           - name: Launch the installed console script from a non-repo dir
238             run: |
239               cd /tmp
240               indexer --help
241               python -c "import backend, training, backend.utils.app_home; print('wheel
import OK')"
242
```

19. user (2026-07-08T05:46:17.482Z)

```
1       [pytest]
2       # When `pytest` is invoked without an explicit path, only collect from tests/.
```

```
3       # scratch/ holds debugging scripts that hit real APIs (Gemini upload, etc.)
4       # and should never run as part of the regular test suite.
5       testpaths = tests
6
```

20. assistant (2026-07-08T05:46:28.210Z)

CI gates captured. Now examining the test files for the touched areas.

21. user (2026-07-08T05:46:32.487Z)

```
=== test_upload_gcs_streaming.py (271 lines) ===
"""GCS-streamed upload staging (CONTROL_PLANE_DURABLE_REBUILD P2 ? the #480/#471 root cause).

Contract under test:
- with a remote input backend (storage.input_backend=gcs + input_root), upload parts STREAM
  through storage.open_writer into <input_root>/uploads/<upload_id>/<name> ? the assembled
  file never exists on local disk, and the free-space guard is not consulted
- /complete returns the staged gs:// URI as `path` (submittable to /api/process verbatim ?
  it is under input_root, inside the #465 jail) and the STREAMED md5 as `video_id`
  (bit-identical to compute_video_id: same bytes, same algorithm)
- abort and the 413 overflow paths finalize-then-DELETE the partial object
- a backend whose open_writer raises NotImplementedError falls back to the local flow
- GCSStorage.open_writer streams via blob.open("wb", ignore_flush=True)
"""

import hashlib
import os

import pytest

pytest.importorskip("httpx")  # TestClient needs httpx; skip cleanly where absent
from fastapi.testclient import TestClient

import backend.main as m
import backend.storage as storage_pkg
from backend.api import uploads as uploads_api

=== test_upload_download.py (372 lines) ===
"""Chunked upload (B2) + highlight download (B3) ? PR-2 of the UI alpha plan.

Contract under test:
- POST /api/upload/init ? {upload_id}; extension allowlist + size caps enforced
- PUT /api/upload/{id}/part/{n} ? strictly sequential parts; per-part/total caps abort (413)
- POST /api/upload/{id}/complete ? assembled file, MD5 == compute_video_id, no overwrite
- DELETE /api/upload/{id} ? abort removes partial state
- GET /api/highlights/{video_id}/{filename} ? streams from output_dir; traversal-proof
"""

import os
from unittest.mock import patch

import pytest

pytest.importorskip("httpx")  # TestClient needs httpx; skip cleanly where absent
from fastapi.testclient import TestClient

import backend.main as m
from backend.api import uploads as uploads_api
from backend.config.models import AppConfig
from backend.infrastructure.database import Database
from backend.utils.hashing import compute_video_id
```

```
=== test_api_cloud_dispatch.py (1105 lines) ===
"""API cloud-serving dispatch (COMPUTE_DECOUPLED_SERVING M6 ? the API path).

Proves the /api/process job path can offload DETECT to a Cloud Run GPU Job and ingest the rally
intervals back into the API DB, so /api/query + /api/compile (local stitch) are unchanged ? all
GPU-free + cloud-free via a FAKE CloudRunJobClient that writes report.json + intervals.csv + the
done-marker into a local "bucket". Cardinal invariant: DEFAULT-OFF ? only
compute_target=cloud_run
dispatches; every other target runs the in-process segmenter byte-identically.
"""

import csv
import json
import os

import pytest

pytest.importorskip("httpx")
from fastapi.testclient import TestClient

import backend.compute as compute_pkg
import backend.main as m
from backend.compute.cloud_run import CloudRunJobClient
from backend.config.models import AppConfig
from backend.infrastructure.database import Database
from backend.infrastructure.execution_policy import ExecutionPolicy
from backend.pipeline.validators.base import ValidationResult

=== test_compute_backend.py (1102 lines) ===
"""ComputeBackend seam ? Cloud Run dispatch shim (COMPUTE_DECOUPLED_SERVING M6).

All GPU-free + cloud-free: the Cloud Run trigger is a FAKE ``CloudRunJobClient`` that simulates
the
worker (writes the done-marker + report to a local "bucket" = a tmp dir), so the
dispatch?bucket-poll
?read flow is proven for $0. The cardinal invariant: **default-OFF** ? only
``compute_target=cloud_run``
selects a backend; everything else returns ``None`` (= run the worker's existing in-process
path).
"""

import json
import os
import tempfile

import pytest

from backend.compute import (
    ComputeJobSpec,
    RemoteExecutionFailed,
    get_compute_backend,
)
from backend.compute.cloud_run import CloudRunCompute, CloudRunJobClient
from backend.config import AppConfig
from backend.infrastructure.execution_policy import ExecutionPolicy, PolicyTimeout

## A fast, no-wait policy (the fake client writes the marker synchronously, so the first pol

_FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)

=== test_validators_base.py (129 lines) ===
"""Tests for the validation registry, focused on the backward-compatible
`run_all_with_context` added for observability reporting."""

from backend.config.models import AppConfig
```

```
from backend.pipeline.validators.base import (
    ValidationRegistry,
    ValidationResult,
    VideoValidator,
)


class _ProducerValidator(VideoValidator):
    @property
    def name(self):
        return "producer"

    @property
    def description(self):
        return "writes ffprobe_meta into the shared context"

    def validate(self, video_path, config, context):
        context["ffprobe_meta"] = {"fps": 59.94, "width": 1920, "height": 1080}
        return ValidationResult(validator_name=self.name, passed=True, message="ok")



=== test_validation_cache.py (284 lines) ===
"""Validation-result cache keyed by video_id (task 4585aa00).

``video_id`` is the MD5 of the file bytes (backend/utils/hashing.py), so an identical-content
REUPLOAD keys the same cache row. When that content was already validated under the SAME
validator
configuration, ``_execute_processing`` reuses the stored verdict instead of re-running the
(minutes-long on a heavy 4K clip) validator suite. This is distinct from the pre-existing
whole-pipeline cache, which only short-circuits on a prior SUCCESSFUL segmentation ? so a
reupload
after a SEGMENTATION FAILURE used to re-pay the full validation from scratch (the 2026-07-07
CUJ).

These tests drive ``_execute_processing`` with a stubbed validator suite (a spy that COUNTS how
many
times it actually runs) + a stubbed segmenter, so the assertions are on validation-run count,
not on
real decoding. Cardinal guarantees exercised: same content + same config ? validators run ONCE
across
two uploads; a validation-config change forces a re-run; ``force=true`` bypasses; a cached FAIL
is
replayed without a re-decode; a different content never reuses another video's verdict.
"""

import logging

import backend.main as m
from backend.config.models import AppConfig
from backend.infrastructure.database import Database
from backend.pipeline.validators.base import ValidationResult, registry
from backend.results import Failure

_ORCH_LOGGER = "backend.orchestration"

=== test_config.py (342 lines) ===
"""Basic tests for the typed configuration package.

These ensure:
- Defaults are sensible and come from the models.
- File overrides work.
- Old config files with extra keys are tolerated.
- save_default_config produces loadable output.
"""
```

```
import json
import os
import tempfile

import pytest

from backend.config import AppConfig, load_config, save_default_config


def test_load_default_config():
    # Force load without the project's config.json so we test the model built-in defaults
    cfg = load_config("/nonexistent/path/that/does/not/exist/12345/config.json")
    assert isinstance(cfg, AppConfig)
    assert cfg.sport == "badminton"
    assert cfg.indexing.default_segmenter == "yolo_hybrid"
    assert cfg.validation.min_blur_threshold == 20.0  # updated default from review

=== test_config_deployment.py (427 lines) ===
"""Tests for the deployment-profile config (COMPUTE_DECOUPLED_SERVING M1).

What this pins:
- The default profile ("") applies NO preset ? the loaded config is byte-identical to the
  pre-M1 default, so the core is unaffected (the project's "default-OFF, zero core change").
- Each profile applies its coherent preset bundle (compute_target + the existing
  detector_impl / skip_ai_handoff / storage.backend knobs).
- Precedence: built-in defaults < profile preset < config.json (a file sub-key overrides a
  preset sub-key while sibling preset/default sub-keys survive ? i.e. real deep-merge layering).
- D15: auto-detect SUGGESTS but never auto-applies (a bare cloud-like env does NOT silently
  select a profile / spend on cloud).
- The model's Literal enum stays in parity with backend/config/presets.PROFILE_PRESETS.
"""

import json
import logging
import os
import sys

import pytest
from pydantic import ValidationError

from backend.config import AppConfig, DeploymentConfig, load_config, save_default_config
from backend.config.presets import (
    COMPUTE_TARGET_FOR_PROFILE,

=== test_submit_md5_cache.py (466 lines) ===
"""Submit-time MD5 + completed-work cache-reuse (#484 ? CONTROL_PLANE_DURABLE_REBUILD P4).

Contract under test:
- a gs:// submit resolves the video's md5 from OBJECT METADATA (one stat, no bytes) and
  persists it on the job row at CREATE (the restart-recovery keystone, spec §4.2)
- when detection already exists ? DB first, bucket `outputs/<md5>/report.json` second ?
  /api/process completes the job INSTANTLY: no drain kick, no GPU dispatch ("MD5 ? instant")
- the bucket branch re-hydrates via the SAME ingest the real cloud path uses (watermarks +
  finalize), so a retry replaces rows rather than double-counting them (#485 mechanism)
- `force=true` bypasses the cache but still records the md5; no-md5 / failed-report inputs
  degrade to the normal queued dispatch
- GCS md5Hash (base64) ? compute_video_id (hex) conversion is exact
"""

import base64
import hashlib
import json
import os
import threading
import time
```

```
import types
from concurrent.futures import ThreadPoolExecutor

import pytest


=== test_worker.py (945 lines) ===
"""Worker dispatcher: pull (storage) ? run pipeline ? push (storage).

`cmd_infer` is exercised end-to-end over the LOCAL storage backend with the validator +
segmenter mocked, so it needs no real video/GPU/network. The real S3 round-trip is validated
on the droplet against MinIO.
"""

import json

import pytest

from backend.config import load_config
from backend.results import Failure
from backend.worker import __main__ as wmain
```

---------- pure helpers

```
@pytest.mark.parametrize(
    "raw,expected",
    [
        ("true", True),
        ("False", False),
        ("3", 3),
        ("2.5", 2.5),
```

22. user (2026-07-08T05:46:47.188Z)

```
tests\test_api_endpoints.py:851:    from backend import orchestration
tests\test_api_endpoints.py:875:    from backend import orchestration
tests\test_api_cloud_dispatch.py:1063:    """The in-process segmenter holds
orchestration._LOCAL_COMPUTE_LANE while it runs, so raising the
tests\test_api_cloud_dispatch.py:1066:    import backend.orchestration as orch
tests\test_api_cloud_dispatch.py:1100:    from backend.orchestration import
_wait_progress_message
tests\test_main_module_budget.py:5:``backend/orchestration.py``; this ratchet keeps the split
from silently regressing:
tests\test_main_module_budget.py:30:# a module instead (orchestration ->
backend/orchestration.py; server/launcher -> backend/server_runtime.py;
tests\test_main_module_budget.py:59:        f"god-module back ? extract to
backend/orchestration.py / backend/api/ / a new "
tests\test_main_module_budget.py:98:    import backend.orchestration as o
tests\test_main_module_budget.py:102:            f"backend.main.{name} is not the
backend.orchestration.{name} object ? "
tests\test_profile_parity.py:4:windows + the same segment ranges** regardless of the active
deployment profile / orchestration.
tests\test_restart_recovery.py:294:    from backend import orchestration
tests\test_restart_recovery.py:312:    from backend import orchestration
tests\test_submit_md5_cache.py:33:from backend.orchestration import cached_process_result,
submit_time_video_id
tests\test_submit_md5_cache.py:424:    from backend import orchestration as orch
tests\test_upload_download.py:296:    from backend.orchestration import reel_object_uri
tests\test_validation_cache.py:25:_ORCH_LOGGER = "backend.orchestration"
```

23. user (2026-07-08T05:46:50.348Z)

```
=== test_api_endpoints.py (1175 lines) ===
```

```
"""TestClient coverage for the FastAPI seam (previously zero ? the compiler SyntaxError
that broke `import backend.main` shipped green because nothing imported it)."""

import os
import subprocess
import sys
from unittest.mock import MagicMock, patch

import pytest

pytest.importorskip("httpx")  # TestClient needs httpx; skip cleanly where absent
from fastapi.testclient import TestClient

import backend.main as m

=== test_restart_recovery.py (527 lines) ===
"""Restart-safe remote jobs + read-path re-hydrate (#471 / #485 ? CP-rebuild P3).

The 2026-07-05 loss shape under test: a running cloud detection's done-marker poll lived only
in process memory; a restart orphaned the job while the L4 worker kept computing, and its 29
rallies landed in the bucket with no listener. Contract now:

- ``recover_stale_jobs`` leaves RUNNING md5-keyed process jobs alone (everything else keeps
  today's semantics: compiles re-queue, other process rows fail terminally)
- ``resume_interrupted_remote_jobs`` re-arms the durable done-signal poll
  (``outputs/<md5>/report.json``) for those rows ? completing or failing them honestly,
  NEVER re-dispatching (no double GPU spend); on a box that can't dispatch to Cloud Run the
  rows get today's terminal failure
- ``POST /api/query`` re-hydrates an md5-shaped id from the bucket before reporting nothing
  (#485: DB = cache, bucket = truth), restoring the video row's truthful source path from the

=== test_drain_concurrency.py (94 lines) ===
"""Drain-executor concurrency (#466).

The in-process job drain is serial (max_workers=1) for truly-local ? a single box serializes GPU
work and the Gemini provider is rate-fragile. Under cloud_run the box only DISPATCHES + bucket-
polls
(I/O-bound; GPU is on ephemeral L4s, WASB-only has no Gemini), so it scales to
``deployment.max_concurrent_remote_jobs``. The worker count is fixed at first executor creation
from
the config (``configure_drain``); everything else stays byte-identical to the serial default.
"""

import pytest
from pydantic import ValidationError

import backend.api.job_worker as jw
from backend.config.models import AppConfig

=== test_async_jobs.py (712 lines) ===
"""Async job model (DEFERRED_HARDENING_PLAN #16).

Contract under test:
- POST /api/process ? 202 + job_id (fast-fail 4xx still happens at submit time)
- GET /api/jobs/{id} ? {status, progress, result, reports, ...}
- job `status` = EXECUTION state (queued|running|done|failed); the pipeline's own
  verdict (e.g. failed validation) lives in result["status"].
- worker crash restores pre-run segments (destroy-AFTER-confirm) and marks the job failed
- startup sweep (fail_stale_jobs) terminalizes jobs orphaned by a restart
- migration: fresh DB lands at user_version 2 with a jobs table; v1 DBs upgrade in place
"""

import json
import sqlite3
```

```
=== test_video.py (46 lines) ===
import subprocess
from unittest.mock import patch

from backend.utils.video import extract_video_chunk, get_video_duration


@patch("subprocess.check_output")
def test_get_video_duration_success(mock_check_output):
    mock_check_output.return_value = b"123.45\n"
    dur = get_video_duration("dummy.mp4")
    assert dur == 123.45
    assert mock_check_output.call_args.kwargs["timeout"] == 60.0



=== test_upload_retention.py (100 lines) ===
import os

from tools import upload_retention


def _touch(path, data=b"x", mtime=0.0):
    path.write_bytes(data)
    os.utime(path, (mtime, mtime))
    return path


def test_plan_upload_retention_only_returns_stale_uploads(tmp_path):
    uploads = tmp_path / "uploads"
    uploads.mkdir()

=== test_deploy_manifests.py (103 lines) ===
"""Structural guard for the control-plane service-manifest generator (#473).

CI can't `gcloud run services replace` anything, so ? like the Dockerfile guard in
test_compute_backend.py ? lock the manifest's load-bearing shape structurally: the baked
config.json must round-trip through the base64 wrapper byte-exactly (that's the piece a
hand-edit silently corrupts), the always-allocated-CPU/scale annotations must survive
(README §3a: without them a 202'd compile stalls), and the reserved `CLOUD_RUN_JOB` env
name must never re-appear (a manifest carrying it is rejected at deploy time).

The generator is exercised through its real CLI (a subprocess), not an import ? deploy/
is not a package, and the CLI is exactly what a deployer runs.
"""

import base64

=== test_execution_policy.py (272 lines) ===
"""High-coverage tests for backend.infrastructure.execution_policy (EXECUTION_POLICY.md P1).

Every branch of run_bounded + poll_until + the ExecutionPolicy (de)serialisation, with injected
clocks so nothing actually waits (hang tests use a tiny real op_timeout ? abandoned daemon
threads
are harmless). Logging discipline (polls at DEBUG, transitions at INFO) is asserted via caplog.
"""

import logging
import time

import pytest

from backend.infrastructure.execution_policy import (
    DEFAULT_POLICY,
```

24. user (2026-07-08T05:46:57.768Z)

```
tests\test_api_endpoints.py:17:from backend.pipeline.validators.base import ValidationResult
tests\test_api_cloud_dispatch.py:25:from backend.pipeline.validators.base import
ValidationResult
tests\test_async_jobs.py:26:from backend.pipeline.validators.base import ValidationResult
tests\test_ffmpeg_timeouts.py:5:from backend.pipeline.validators.format import FormatValidator
tests\test_ffmpeg_timeouts.py:6:from backend.pipeline.validators.pts_continuity import
PTSContinuityValidator
tests\test_frame_sampler.py:30:from backend.pipeline.validators import frame_sampler
tests\test_frame_sampler.py:31:from backend.pipeline.validators.base import ValidationRegistry
tests\test_frame_sampler.py:32:from backend.pipeline.validators.blur import BlurValidator
tests\test_frame_sampler.py:33:from backend.pipeline.validators.frame_sampler import
run_frame_sampling
tests\test_frame_sampler.py:34:from backend.pipeline.validators.relevance import
RelevanceValidator
tests\test_frame_sampler.py:35:from backend.pipeline.validators.scene_cut import
SceneCutDensityValidator
tests\test_frame_sampler.py:437:        "backend.pipeline.validators.base.run_frame_sampling",
flaky
tests\test_job_worker_loop.py:35:from backend.pipeline.validators.base import ValidationResult
tests\test_reporting_harvest.py:13:from backend.pipeline.validators.base import ValidationResult
# noqa: E402
tests\test_validation_cache.py:22:from backend.pipeline.validators.base import ValidationResult,
registry
tests\test_validators_base.py:5:from backend.pipeline.validators.base import (
```

25. user (2026-07-08T05:46:59.109Z)

```
__init__.py
__pycache__
base.py
blur.py
format.py
frame_sampler.py
pts_continuity.py
relevance.py
resolution_fps.py
scene_cut.py
```

26. user (2026-07-08T05:47:09.471Z)

```
"""Regression suite for the shared single-pass frame sampler (perf fix).

Context: scene_cut / blur / relevance each used to open their OWN cv2.VideoCapture
and pull sample frames with per-sample ``cap.set(CAP_PROP_POS_FRAMES, idx)`` seeks
? ~125 random seeks across THREE decode opens, which cost minutes on long-GOP 4K
GoPro clips on the CPU-only control-plane. They now share ONE monotonic-forward
decode pass (backend/pipeline/validators/frame_sampler.py).

These tests pin the behaviour that makes that safe:
  * each validator's ``sample_indices`` == the exact legacy per-seek arithmetic;
  * the shared pass opens the capture ONCE and retrieves EXACTLY the union of
    those indices, in monotonic-forward order, with far fewer seeks than samples;
  * frames are dispatched to the right validator for the right index;
  * pass/fail decisions and the standalone fallback are unchanged.

They use an in-memory FakeCapture (no real media file, no codec), so they run in
milliseconds and are covered by CI (unlike the concrete validators, which are
coverage-omitted as real-media-only wrappers).
"""

from __future__ import annotations

import functools
```

```
import cv2
import numpy as np
import pytest

---- config coverage of deployment/phase knobs ----
6:- Each profile applies its coherent preset bundle (compute_target + the existing
50:    assert cfg.deployment.compute_target == ""
70:        "compute_target": "",
71:        "max_concurrent_remote_jobs": 4,
90:def test_compute_target_literal_matches_presets():
94:        typing.get_args(DeploymentConfig.model_fields["compute_target"].annotation)
119:    assert cfg.deployment.compute_target == "cpu_mock"
128:    assert cfg.deployment.compute_target == "cloud_run"
139:    assert cfg.deployment.compute_target == "local_gpu"
186:def test_config_json_can_override_compute_target_with_warning(tmp_path, caplog):
190:            "deployment": {"profile": "cpu-mock", "compute_target": "cloud_run"},
197:        cfg.deployment.compute_target == "cloud_run"
202:def test_empty_compute_target_under_profile_backfills_to_canonical(tmp_path):
203:    """An active profile whose compute_target is blanked to "" in config.json gets its
canonical
204:    target back ? the record never silently records an active profile with no
compute_target."""
209:                "deployment": {"profile": "cloud-serving", "compute_target": ""},
214:    assert cfg.deployment.compute_target == "cloud_run"
233:    assert cfg.deployment.compute_target == "cpu_mock"  # cpu-mock preset applied
263:    assert cfg.deployment.compute_target == "cpu_mock"  # preset fully applied
287:    assert cfg.deployment.compute_target == ""
397:        "compute_target": "",
398:        "max_concurrent_remote_jobs": 4,
```

27. user (2026-07-08T05:47:10.111Z)

```
146:def test_cloud_dispatch_ingests_segments(cloud_env):
165:def test_cloud_dispatch_builds_correct_spec(cloud_env):
189:def test_cloud_dispatch_forwards_skip_ai_handoff_true(cloud_env):
202:def test_cloud_dispatch_forwards_wasb_serving_knobs(cloud_env):
222:def test_cloud_dispatch_omits_unset_wasb_tuning(cloud_env):
239:def test_cloud_dispatch_skips_worker_revalidation(cloud_env):
255:def test_cloud_failure_marks_failed_no_rows(cloud_env):
275:def test_cloud_requires_a_bucket_input(cloud_env, tmp_path):
289:def test_cloud_stages_local_upload_to_bucket(cloud_env, tmp_path, monkeypatch):
330:def test_cloud_stages_when_input_root_is_cloud_and_input_backend_default(
367:def test_cloud_local_upload_without_input_root_still_fails(cloud_env, tmp_path,
monkeypatch):
384:def test_default_off_runs_in_process(tmp_path, monkeypatch):
425:def test_cloud_ingest_failure_marks_failed_no_leak(cloud_env):
454:def test_cloud_video_id_divergence_is_tolerated_with_warning(cloud_env, caplog):
493:def test_per_request_local_forces_in_process(cloud_env):
516:def test_per_request_cloud_forces_dispatch_on_local_server(tmp_path, monkeypatch):
558:def test_per_request_cloud_unconfigured_fails_clearly(tmp_path, monkeypatch):
586:def test_unknown_compute_value_falls_back_to_server_default(cloud_env, caplog):
605:def test_capabilities_reports_cloud_available(cloud_env):
617:def test_capabilities_cloud_unavailable_by_default(monkeypatch, tmp_path):
630:def test_full_api_process_to_query_via_cloud(cloud_env):
658:def test_local_override_survives_api_boundary(cloud_env):
691:def test_cloud_override_survives_api_boundary_on_local_server(tmp_path, monkeypatch):
747:def test_local_on_cloud_server_without_weights_fails_fast(cloud_env, monkeypatch):
772:def test_local_on_cloud_server_with_weights_present_runs_in_process(
796:def test_local_non_weights_segmenter_on_cloud_server_still_runs_in_process(
817:def test_local_weights_backed_on_pure_local_server_is_not_guardrailed(
848:def test_local_tracknet_on_cloud_server_without_weights_fails_fast(cloud_env, monkeypatch):
868:def test_local_fusion_tracknet_substrate_checks_tracknet_not_wasb(
894:def test_local_fusion_default_substrate_with_wasb_weights_runs_in_process(
922:def test_cloud_dispatch_stamps_compute_provenance(cloud_env):
944:def test_local_override_stamps_in_process_provenance(cloud_env):
```

```
960:def test_default_in_process_stamps_provenance(tmp_path, monkeypatch):
981:def test_cloud_requested_unconfigured_stamps_not_dispatched(tmp_path, monkeypatch):
1018:def test_validation_failure_stamps_not_run_provenance(tmp_path, monkeypatch):
1036:def test_full_api_process_job_result_carries_cloud_provenance(cloud_env):
1062:def test_in_process_detection_runs_under_the_local_lane(cloud_env):
1097:def test_wait_progress_message_is_honest_and_minute_granular():
```

28. user (2026-07-08T05:47:22.267Z)

```
16:``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
648:        # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
748:        local_video = storage.acquire_local_input(
751:        video_id = _main.compute_video_id(local_video)
1159:         local_src = storage.acquire_local_input(
```

29. user (2026-07-08T05:47:23.711Z)

```
tests\test_api_cloud_dispatch.py:245:    carry --skip-validation, telling the worker to trust
the gate. Regression guard for that wiring."""
tests\test_api_cloud_dispatch.py:252:    assert "--skip-validation" in fake.calls[0]
tests\test_compute_backend.py:256:    # A DEFAULT spec (skip_validation=False) must NOT carry
--skip-validation: the worker still
tests\test_compute_backend.py:257:    # validates unless the dispatcher explicitly opts out.
Guards the `if spec.skip_validation:` gate
tests\test_compute_backend.py:259:    # `worker infer` cloud path's validation ? the very thing
#skip-validation is scoped away from).
tests\test_compute_backend.py:260:    assert "--skip-validation" not in argv
tests\test_compute_backend.py:272:def
test_dispatch_argv_carries_skip_validation_when_set(tmp_path):
tests\test_compute_backend.py:273:    """The builder must emit --skip-validation when the spec
opts out ? the control-plane sets
tests\test_compute_backend.py:274:    skip_validation=True on every dispatch (it is the
authoritative validation gate), and the worker
tests\test_compute_backend.py:285:            skip_validation=True,
tests\test_compute_backend.py:288:    assert "--skip-validation" in client.calls[0]["argv"]
tests\test_worker.py:100:def test_parser_infer_skip_validation_flag():
tests\test_worker.py:103:    assert wmain.build_parser().parse_args(base).skip_validation is
False
tests\test_worker.py:105:    assert wmain.build_parser().parse_args(base + ["--skip-
validation"]).skip_validation is True
tests\test_worker.py:418:def test_cmd_infer_skip_validation_bypasses_validators(tmp_path,
monkeypatch):
tests\test_worker.py:419:    """--skip-validation (set by the dispatcher, the authoritative
gate) must NOT run the worker's
tests\test_worker.py:427:        raise AssertionError("run_all_with_context must not run under
--skip-validation")
tests\test_worker.py:447:            "--skip-validation",
```

30. user (2026-07-08T05:47:25.169Z)

```
(Bash completed with no output)
```

31. user (2026-07-08T05:47:34.896Z)

```
64:# SHA; fetched over HTTPS. If you lack access, simply omit this group ? reporting
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/.coveragerc
[run]
```

Excluded from coverage: the legacy motion segmenter (demo baseline with fabricated metadata) and the concrete validators (thin OpenCV/ffprobe wrappers only exercisable against real media files). validators/base.py stays counted ? it has a dedicated unit suite.

```
omit =
    backend/pipeline/segmenters/motion.py
    backend/pipeline/validators/format.py
    backend/pipeline/validators/resolution_fps.py
    backend/pipeline/validators/pts_continuity.py
    backend/pipeline/validators/scene_cut.py
    backend/pipeline/validators/blur.py
    backend/pipeline/validators/relevance.py
```

32. user (2026-07-08T05:47:44.199Z)

```
735             config = _main.global_config
736         if db is None:
737             db = _main.db_instance
738
739         notify = progress or (lambda stage: None)
740
741         notify("hashing")
742         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
743         # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
744         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
745         # consumers (hash / validators / segmenter) use the resolved local path.
746         local_video = ""
747         try:
748             local_video = storage.acquire_local_input(
749                 req.video_path, getattr(config, "storage", None)
750             )
751             video_id = _main.compute_video_id(local_video)
752         except Exception:
753             storage.cleanup_acquired_local_input(
754                 req.video_path, local_video, getattr(config, "storage", None)
755             )
756             raise
757         try:
758             existing_video = db.get_video(video_id)
759         except Exception:
760             storage.cleanup_acquired_local_input(
761                 req.video_path, local_video, getattr(config, "storage", None)
762             )
763             raise
764         was_reingest = existing_video is not None
765
766         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
767         default_seg = getattr(
768             getattr(config, "indexing", None), "default_segmenter", "motion"
769         )
```

33. user (2026-07-08T05:47:45.623Z)

```
1:"""GCS-streamed upload staging (CONTROL_PLANE_DURABLE_REBUILD P2 ? the #480/#471 root cause).
34:    """Records streamed bytes; materializes the object in `store` on close."""
107:def test_remote_parts_stream_to_bucket_with_md5_identity(env):
122:    # The object holds exactly the streamed bytes and the writer was finalized.
129:def test_remote_streamed_md5_matches_compute_video_id(env, tmp_path):
130:    # The streamed digest must be bit-identical to hashing the same bytes from disk ?
```

34. user (2026-07-08T05:47:57.782Z)

```
=== test_job_worker_loop.py (359 lines) ===
"""P3 worker self-scheduling loop (EXECUTION_POLICY.md P3).
```

```
The P2 slice added the DB primitives (`claim_due_job`, `set_job_waiting`,
`seconds_until_next_due`); this slice wires the `main.py` worker to them so the worker
DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job that raises
`JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and re-claimed when
due ? instead of the old one-shot `submit(_run_job, ...)`.

Covered:
- `_drain_due_jobs` claims + runs queued jobs to a terminal state (no regression on the
  existing sync/submit path ? same `done`/`failed`/verdict semantics);
- earliest-due ordering across queued + due-waiting;

=== test_hashing.py (50 lines) ===
"""Tests for the canonical video_id hashing helper.

Guards the de-duplication of the former 4 divergent get_file_md5 copies
(backend/main.py + the three run_*.py scripts) into one shared function.
"""

import hashlib
import os
import tempfile

from backend.utils.hashing import compute_video_id


=== test_startup_checks.py (518 lines) ===
"""Per-profile startup checks (COMPUTE_DECOUPLED_SERVING M5).

L1 unit coverage of ``backend/config/startup.py``: the fail/warn-fast per-profile checks.
Pure ? no torch/GPU/network (the CUDA probe + env are injected). The cardinal invariant:
**no profile selected ? ZERO checks** (default-OFF, identical to pre-M5).
"""

import logging

import pytest

from backend.config import AppConfig

98:def cloud_env(tmp_path, monkeypatch):
```

35. assistant (2026-07-08T05:48:45.891Z)

```
I have everything needed. Compiling the structured map.
```

36. user (2026-07-08T05:50:34.765Z)

```
Structured output provided successfully
```